

Tài liệu chi tiết cho dự án RAE XLA và JAX

Nhánh TorchXLA/TPU và lớp tương thích JAX/NNX của dự án Representation Autoencoders

Ngày 17 tháng 4 năm 2026

Mục lục

1	Tổng quan dự án	3
1.1	Mục tiêu	3
1.2	Hai giai đoạn chính	3
1.2.1	Stage 1	3
1.2.2	Stage 2	3
1.3	Phạm vi của branch hiện tại	3
1.4	Giới hạn hiện tại	3
1.5	Dòng dữ liệu cấp cao	4
1.6	Ý tưởng của đường JAX	4
1.7	Tài liệu trong repo	4
2	Kiến trúc hệ thống	4
2.1	Bản đồ codebase	4
2.2	Stage 1: Representation Autoencoder	5
2.3	Stage 2: SiT trong không gian latent	5
2.4	Transport objective	6
2.5	Lớp adapter JAX	6
2.6	Hai mô hình thực thi	7
2.6.1	TorchXLA	7
2.6.2	JAX / NNX	7
2.7	Checkpoint và tương thích trọng số	7
3	Workflow thực chiến	8
3.1	Cài đặt môi trường	8
3.1.1	Đường XLA	8
3.1.2	Phần bổ sung cho JAX / NNX	8
3.2	Chuẩn bị model và dữ liệu	8
3.2.1	Model	8
3.2.2	Dữ liệu	8
3.3	Kiểm tra Stage 1 bằng reconstruction	9
3.3.1	Đường XLA / PyTorch	9
3.3.2	Đường JAX	9
3.3.3	Tính latent stat cho Stage 1 trên đường JAX	9
3.3.4	Reconstruction cả thư mục trên đường JAX	9
3.4	Huấn luyện Stage 1 trên local GPU	10

3.5	Huấn luyện Stage 2 trên TPU bằng TorchXLA	10
3.6	Huấn luyện Stage 2 bằng JAX / NNX	11
3.7	Wandb logging	11
3.8	FID trong train loop	12
3.9	Sampling Stage 2	12
3.9.1	JAX một lệnh	12
3.9.2	JAX distributed	13
3.10	Đẩy artifact lên Hugging Face	13
3.10.1	Lệnh độc lập	13
3.10.2	Tích hợp ngay trong train	13
3.11	Notebook Kaggle TPU cho CelebA	13
4	Cấu hình và ánh xạ sang JAX	15
4.1	Các block config chính	15
4.2	Ma trận script và block config	15
4.3	Block stage_1	16
4.4	Block stage_2	16
4.5	Block guidance	17
4.6	Block training	17
4.7	Block gan	18
4.8	Block data	18
4.9	Block source	18
4.10	Block eval	19
4.11	Override từ CLI	19
5	Vận hành, artifact và FID	20
5.1	Sampling Stage 2 trên đường JAX	20
5.1.1	Sampling nhanh	20
5.1.2	Sampling phân tán	20
5.2	Build reference statistics cho FID	20
5.3	Artifact của Stage 2 training	21
5.3.1	Đường XLA	21
5.3.2	Đường JAX	21
5.4	Đẩy model lên Hugging Face	21
5.5	Các lưu ý vận hành quan trọng	21
5.6	Checklist thực tế trước khi chạy dài	22

1. Tổng quan dự án

1.1. Mục tiêu

Dự án này triển khai pipeline tạo ảnh hai giai đoạn dựa trên *Representation Autoencoders* (RAE). Ở thời điểm hiện tại, repo có hai đường chạy chính:

- `src/`: nhánh TorchXLA, tối ưu cho TPU với train và sampling Stage 2, reconstruction Stage 1, và FID phía host;
- `src_jax/`: lớp tương thích JAX/NNX, giữ nguyên schema YAML của repo nhưng ánh xạ sang backend NNX để tránh nhân đôi toàn bộ codebase.

1.2. Hai giai đoạn chính

1.2.1. Stage 1

Stage 1 dùng một encoder biểu diễn đã được pretrain và đóng băng, kết hợp với một decoder ViT để tái tạo ảnh. Đầu ra của Stage 1 là latent giàu ngữ nghĩa hơn latent VAE thông thường.

1.2.2. Stage 2

Stage 2 học mô hình diffusion transformer trong không gian latent của Stage 1. Thay vì học trực tiếp trên pixel, mô hình học quy luật biến đổi của latent, sau đó giải mã latent sinh được thành ảnh RGB qua decoder của RAE.

1.3. Phạm vi của branch hiện tại

Nhánh hiện tại không chỉ có đường TorchXLA mà còn có adapter JAX mỏng. Phần được hỗ trợ thực dụng nhất là:

- huấn luyện Stage 2 bằng `src/train.py` trên TPU;
- huấn luyện Stage 2 bằng `src_jax/train.py` trên JAX/NNX;
- sampling Stage 2 bằng cả `src/` và `src_jax/`;
- reconstruction Stage 1 để kiểm tra encoder-decoder trên cả hai đường;
- logging qua wandb;
- upload artifact hoặc workdir lên Hugging Face;
- FID offline, và online FID trong train loop ở đường XLA.

1.4. Giới hạn hiện tại

- `src/` vẫn không có endpoint riêng cho huấn luyện Stage 1 trên XLA;
- `src_jax/` cố ý chưa port toàn bộ vòng huấn luyện Stage 1 kiểu GAN + LPIPS + discriminator, vì phần đó sẽ làm codebase lớn và khó bảo trì hơn đáng kể.

1.5. Dòng dữ liệu cấp cao

Bước	Mô tả
1	Ảnh đầu vào được đưa qua encoder của Stage 1
2	Encoder sinh latent tensor có kích thước như <code>misc.latent_size</code>
3	Stage 2 học transport objective trên latent đó
4	Khi sampling, latent được sinh bởi Stage 2
5	Decoder của Stage 1 giải mã latent thành ảnh RGB

1.6. Ý tưởng của đường JAX

Đường JAX không tạo thêm một schema config mới. Thay vào đó:

- vẫn đọc các block `stage_1`, `stage_2`, `transport`, `sampler`, `guidance`, `misc`, `training`, `eval`;
- dùng `src_jax/config_adapter.py` để chuyển YAML hiện có sang config mà backend NNX hiểu được;
- bootstrap backend `diffuse_nnx` theo commit cố định để branch gọn hơn, không phải chép toàn bộ framework vào repo.

1.7. Tài liệu trong repo

Ngoài PDF này, repo còn có bộ tài liệu markdown trong `docs/`. Phần markdown thiên về runbook ngắn và tra cứu nhanh; PDF này gom lại kiến trúc, workflow, cấu hình và vận hành cho cả đường XLA lẫn JAX.

2. Kiến trúc hệ thống

2.1. Bản đồ codebase

```
configs/
  stage1/
  stage2/
src/
  stage1/
  stage2/
  utils/
  train_stage1_rae.py
  train.py
  sample.py
  sample_ddp.py
  stage1_sample.py
  stage1_sample_ddp.py
  build_fid_stats.py
  evaluate_fid.py
src_jax/
  backend_overlay/
  vendor.py
  moe1/
  config_adapter.py
  stage2_runtime.py
  stage1_runtime.py
```

```

build_stage1_stats.py
build_source_gmm.py
export_celebahq_hf.py
export_celebahq_tfds.py
reconstruct_folder.py
hf_utils.py
train.py
sample.py
sample_ddp.py
stage1_sample.py
push_hf.py
raes-jax-celeba-kaggle-tpuv5e8-sitdh-b-moe1.ipynb
raes-jax-celeba-kaggle-tpuv5e8-sitdh-b-moe1-resume.ipynb
tunedinov2-stage1-scratch-kaggle.ipynb
tunedinov2-stage1-finetune-dinov2-kaggle.ipynb

```

2.2. Stage 1: Representation Autoencoder

Thành phần gốc của Stage 1 nằm trong `src/stage1/rae.py`. Lớp RAE kết hợp:

- encoder ở `src/stage1/encoders/`,
- decoder ở `src/stage1/decoders/`,
- latent normalization qua `normalization_stat_path`,
- latent noising qua `noise_tau`.

Hành vi chính:

- `encode(x)` resize đầu vào về đúng độ phân giải của encoder, chuẩn hóa theo image processor, chạy encoder, reshape latent về dạng 2D nếu cần, rồi áp normalization;
- `decode(z)` đảo normalization, đổi latent về dạng sequence nếu cần, rồi giải mã thành ảnh RGB bằng decoder ViT.

Repo giờ cũng có entrypoint `src/train_stage1_rae.py` cho huấn luyện Stage 1 trên local GPU / Kaggle GPU. Script này ghép lại RAE, LPIPS, DINO discriminator, checkpointing, preview PNG, log wandb, torchrun-style DDP, gradient accumulation, và reconstruction FID để phục vụ cả decoder-from-scratch lẫn finetune DINOv2.

Ở đường JAX, `src_jax/stage1_runtime.py` không tự viết lại toàn bộ RAE trong repo này. Nó gọi backend NNX, nhưng vẫn dùng đúng các giá trị cấu hình trong block `stage_1`.

2.3. Stage 2: SiT trong không gian latent

Surface Stage 2 mặc định của branch này nằm ở `src/stage2/models/SiT.py`. Lớp SiTDH là alias repo-facing cho backbone DH/two-tower DiTwDDTHead trong `src/stage2/models/DDT.py`. Điều này giữ cho kiến trúc Stage 2 khớp với biến thể DH, còn objective trên đường JAX vẫn đi qua interface `sit`.

Các đặc điểm quan trọng:

- đầu vào là latent thay vì pixel;
- embedding thời gian bằng Gaussian Fourier features;

- class conditioning bằng LabelEmbedder;
- tùy chọn ROPE, RMSNorm, SwiGLU, qk-norm;
- hỗ trợ classifier-free guidance và autoguidance khi sampling.

Đường JAX ánh xạ `stage_2.target` sang backbone tương ứng của backend NNX:

- SiTDH $\rightarrow$ `lightning_ddt`,
- LightningDiT $\rightarrow$ `lightning_dit`,
- các target DDT legacy $\rightarrow$ `lightning_ddt`,
- các target kiểu DiT khác $\rightarrow$ `dit`.

2.4. Transport objective

Phần này nằm trong `src/stage2/transport/transport.py`. Đây là lớp trung gian giữa latent diffusion model và train loop.

Nhiệm vụ của transport:

- lấy mẫu thời gian t ;
- xây dựng đường nối giữa noise và dữ liệu thật;
- tính target phù hợp với loại output của model;
- trả về drift function để phục vụ ODE hoặc SDE sampler.

Ở đường JAX, adapter không giữ lại implementation transport gốc này. Nó ánh xạ các ý nghĩa tương đương sang interface của backend NNX, chủ yếu là SiT với sampling kiểu Euler.

2.5. Lớp adapter JAX

File	Vai trò
<code>src_jax/vendor.py</code>	Bootstrap <code>diffuse_nnx</code> vào ■/.cache/rae_jax/diffuse_nnx và pin commit
<code>src_jax/config_adapter.py</code>	Chuyển YAML OmegaConf của repo sang config backend NNX, forward random_flip/prefetch_factor và mặc định bật flip cho config train CelebA-HQ
<code>src_jax/stage2_runtime.py</code>	Train, sampling, load checkpoint PyTorch hoặc Orbax, JAX validation-loss integration, raw-image random flip tùy chọn, FID glue, wandb bridge
<code>src_jax/stage1_runtime.py</code>	Load encoder Stage 1 JAX, reconstruction một ảnh, reconstruction cả thư mục và tích lũy latent stat
<code>src_jax/build_stage1_stat.py</code>	Export file stat.pt theo định dạng mean / var để dùng lại cho Stage 1
<code>src_jax/export_celebahq_hq.py</code>	Download eurecom-ds/celeba-hq-256 từ Hugging Face rồi export thành cây ImageFolder cho pipeline JAX hiện tại
<code>src_jax/export_celebahq_tfds.py</code>	Export TFDS celeb_a_hq/256 thành cây ImageFolder cho pipeline JAX hiện tại và ép protobuf runtime kiểu Python để tránh lỗi import TFDS trên Kaggle

File	Vai trò
<code>src_jax/build_fid_stats.py</code>	Build <code>fid_ref</code> bằng đúng detector Flax Inception của backend JAX
<code>src_jax/reconstruct_folder.py</code>	Export reconstruction cho cả thư mục ảnh theo pipeline JAX
<code>src_jax/hf_utils.py</code>	Upload file hoặc thư mục lên Hugging Face

2.6. Hai mô hình thực thi

2.6.1. TorchXLA

Các entrypoint distributed trong `src/` dùng `xmp.spawn(...)` và mỗi TPU core chạy một process riêng. Các primitive quan trọng:

- `DistributedSampler`,
- `ParallelLoader`,
- `xm.optimizer_step(...)`,
- `xm.rendezvous(...)` và `xm.mesh_reduce(...)`.

2.6.2. JAX / NNX

Đường `src_jax/` không tự quản lý toàn bộ graph, checkpoint và sampler ở mức thấp. Nó dựa vào backend NNX để:

- tạo model, optimizer, sampler và EMA,
- quản lý checkpoint Orbx,
- chạy sampling và FID trong định dạng tensor NHWC của JAX,
- mở rộng sang nhiều process JAX nếu môi trường đã cấu hình phù hợp.

Repo cũng vá backend này để EMA khởi tạo từ bản copy của model hiện thời thay vì một cây tham số toàn số 0. Nhờ vậy `ema_network_samples` và online FID không còn bắt đầu từ một EMA cold-start sai ngay từ step đầu.

2.7. Checkpoint và tương thích trọng số

Với branch `jax-sit-dh-moe1`, checkpoint Stage 2 phải khớp shape của SiTDH/LightningDiT:

- nếu `stage_2.ckpt` là file PyTorch `.pt`, adapter sẽ port state dict sang NNX để inference hoặc khởi tạo train nếu checkpoint đó đã tương thích với SiTDH;
- nếu `stage_2.ckpt` là thư mục checkpoint Orbx, adapter sẽ restore trực tiếp run JAX SiTDH trước đó;
- checkpoint DDT legacy không được tự động convert trên branch này;
- decoder Stage 1 ở đường JAX vẫn dùng được checkpoint decoder PyTorch.

3. Workflow thực chiến

3.1. Cài đặt môi trường

Repo hiện có hai cụm phụ thuộc chính.

3.1.1. Đường XLA

```
conda create -n rae python=3.10 -y
conda activate rae
pip install uv
uv pip install torch~=2.5.0 torch_xla[tpu]~=2.5.0 torchvision==0.20.1 -f https://
    storage.googleapis.com/libtpu-releases/index.html
uv pip install timm==0.9.16 accelerate==0.23.0 torchdiffeq==0.2.5 wandb scipy torch-
    fidelity
uv pip install "numpy<2" "transformers==4.57.1" einops
```

3.1.2. Phân bổ sung cho JAX / NNX

```
uv pip install "jax[cuda12]==0.5.1" flax==0.10.4 optax==0.2.4 orbax-checkpoint==0.11.16
uv pip install ml-collections clu absl-py etils huggingface_hub
```

Lần chạy JAX đầu tiên sẽ tự bootstrap backend `diffuse_nnx` vào thư mục `./.cache/rae_jax/diffuse_nnx`. Các notebook JAX trên Kaggle không còn lặp lại từng lệnh `uv pip install` này; chúng đặt `UV_PROJECT_ENVIRONMENT`, `UV_CACHE_DIR=/tmp/uv-cache`, rồi chạy `uv sync -q` theo `pyproject.toml` của repo trước các cell phụ thuộc package. Ở branch này, các notebook Kaggle giờ chỉ bootstrap secret `WANDB2` cho xác thực W&B; chúng không còn đọc `HF_TOK_WRITE_KAGGLE` ngay khi khởi động notebook. `HF_TOKEN` chỉ còn cần khi bạn chủ động bật luồng upload lên Hugging Face như `-hf-repo-id` hoặc `src_jax/push_hf.py`. Repo này tự vá backend `diffuse_nnx` để import các model `Dinov2` từ subpackage của `transformers`, nên nên dùng `transformers==4.57.1` cho đường này. Patch này cũng chuyển `google-cloud-storage` sang import lười, nên đường Stage 1 với RAE/DINO không cần cài package đó trừ khi bạn thực sự dùng các thành phần backend có tải asset từ GCS. Patch bootstrap này cũng sửa khởi tạo EMA của backend để EMA bắt đầu từ bản copy của model hiện thời thay vì cây tham số toàn số 0. Nếu bạn resume một checkpoint Orbax cũ được tạo trước khi có fix này, checkpoint đó vẫn mang EMA đã lưu sẵn trước đó. Adapter cũng tự suy ra `downsample_factor` và `latent_channels` từ `misc.latent_size`, nên bước preview sample và FID của backend vẫn chạy ở latent resolution thay vì lỗi cấp phát noise theo image resolution.

3.2. Chuẩn bị model và dữ liệu

3.2.1. Model

```
hf download nyu-visionx/RAE-collections --local-dir models
```

3.2.2. Dữ liệu

Hầu hết các script dùng dữ liệu dạng `ImageFolder`. Ví dụ:

```
dataset_root/
  class_a/
    0001.png
```

```
class_b/
  0002.png
```

3.3. Kiểm tra Stage 1 bằng reconstruction

3.3.1. Đường XLA / PyTorch

```
python src/stage1_sample.py \
  --config <config> \
  --image assets/pixabay_cat.png \
  --output recon.png
```

3.3.2. Đường JAX

```
python3 src_jax/stage1_sample.py \
  --config <config> \
  --image assets/pixabay_cat.png \
  --output recon_jax.png
```

Lệnh này phù hợp để xác nhận:

- checkpoint decoder Stage 1 có load được hay không,
- latent shape có đúng hay không,
- preprocessing của encoder có khớp hay không.

3.3.3. Tính latent stat cho Stage 1 trên đường JAX

Với dataset mới như CelebA, nên tạo một file bootstrap identity stat trước (mean = 0, var = 1), rồi chạy:

```
python3 src_jax/build_stage1_stats.py \
  --config <stage1_config> \
  --input <train_imagefolder> \
  --output <stage1_stat.pt> \
  --batch-size 16 \
  --set stage_1.params.normalization_stat_path=<bootstrap_identity_stat.pt>
```

File stage1_stat.pt tạo ra vẫn cùng định dạng với repo gốc, nên dùng lại được cho cả đường src/ lẫn src_jax/. Các lệnh Stage 1 thuần ở JAX có thể chạy với YAML chỉ khai báo stage_1; chúng không bắt buộc có stage_2.target.

3.3.4. Reconstruction cả thư mục trên đường JAX

```
python3 src_jax/reconstruct_folder.py \
  --config <stage1_config> \
  --input <val_imagefolder> \
  --output-dir recon_jax_dir \
  --batch-size 8
```

Lệnh này hữu ích khi muốn export reconstruction set trước khi build FID reference hoặc khi cần so sánh decoder Stage 1 giữa các checkpoint.

3.4. Huấn luyện Stage 1 trên local GPU

```
uv run python src/train_stage1_rae.py \
  --config configs/stage1/training/DINOv2-B_decXL.yaml \
  --train-data-path /path/to/train_imagefolder \
  --val-data-path /path/to/val_imagefolder \
  --results-dir results_stage1 \
  --exp-name tunedinov2-stage1 \
  --wandb \
  --wandb-project TuneDinoV2
```

Entry point này dành cho local GPU hoặc Kaggle GPU, không đi qua TPU/XLA. Nó đọc các block `stage_1`, `training`, `gan`, và tùy chọn `data/eval`; ghi checkpoints/`last.pt`, checkpoints/`best.pt`, export preview PNG vào `samples/`, và log L1, LPIPS, GAN loss, latent RMS/variance, learning rate, grad norm, cùng ảnh reconstruction train/val lên wandb. Nếu `training.train_encoder=true`, script sẽ mở train DINOv2 và dùng `training.encoder_lr` riêng cho encoder. Trên host nhiều GPU, cùng entry point này có thể chạy qua `torchrun`; khi đó `training.batch_size` được hiểu là micro-batch mỗi process, còn `training.grad_accum_steps` mở rộng effective batch thêm một lớp nữa.

```
uv run torchrun --standalone --nproc_per_node=2 src/train_stage1_rae.py \
  --config configs/stage1/training/DINOv2-B_decXL.yaml \
  --train-data-path /path/to/train_imagefolder \
  --val-data-path /path/to/val_imagefolder \
  --results-dir results_stage1 \
  --exp-name tunedinov2-stage1-ddp \
  --wandb \
  --wandb-project TuneDinoV2
```

Repo cũng ship sẵn hai notebook Kaggle GPU bọc quanh trainer này: `tunedinov2-stage1-scratch-kaggle.ipynb` cho decoder-from-scratch và `tunedinov2-stage1-finetune-dinov2-kaggle.ipynb` cho finetune DINOv2.

Hai notebook này giờ đều bootstrap uv qua `curl ... | sh`, thêm `./local/bin` vào PATH, mặc định `grad_accum_steps=8`; nếu session Kaggle lộ ra nhiều GPU thì chúng tự chuyển từ `uv run python` sang `uv run torchrun`. Các key tùy chọn của phần eval như `fid_num_threads` có thể để null. Nếu cần refresh lại cả hai notebook từ template, chạy thêm `python3 scripts/generate_tunedinov2_notebooks.py`.

3.5. Huấn luyện Stage 2 trên TPU bằng TorchXLA

```
python src/train.py \
  --config configs/stage2/training/ImageNet256/SiTdH-XL_DINOv2-B.yaml \
  --data-path <imagenet_train_split> \
  --results-dir results \
  --image-size 256 \
  --precision bf16
```

Trong mỗi optimizer step, train loop thực hiện:

1. center crop và đưa ảnh thành tensor;
2. mã hóa ảnh bằng RAE;
3. tính transport loss trong latent space;
4. step optimizer và scheduler trên TPU;

5. update EMA;
6. chạy logging, checkpointing, preview sample, validation và FID theo cadence.

3.6. Huấn luyện Stage 2 bằng JAX / NNX

```
python3 src_jax/train.py \
  --config configs/stage2/training/ImageNet256/SiTdh-XL_DINOv2-B.yaml \
  --data-path <imagenet_train_root> \
  --results-dir results_jax \
  --precision bf16 \
  --wandb
```

Trên Kaggle TPU, bộ notebook JAX của branch này hiện đã chuẩn hoá về `-set training.num_workers=16`, `-set training.prefetch_factor=4`, và `-set eval.prefetch_factor=4`. Mốc này giữ backend PyTorch loader ở vùng ổn định cho notebook JAX đóng gói sẵn, đồng thời vẫn cho host queue trước vài batch để chồng lấp I/O với TPU. Adapter cũng tự tắt backend TensorBoard summary writer trên Kaggle và giữ metric ở stdout cùng wandb.

Điểm quan trọng:

- vẫn dùng cùng file YAML của repo;
- hỗ trợ `-set key=value` để override nhanh từ CLI;
- hỗ trợ `-wandb-run-id <existing_run_id>` để bind một legacy workdir với đúng W&B run cũ đúng một lần;
- nếu `stage_2.ckpt` là file PyTorch `.pt`, adapter sẽ port trọng số sang NNX để khởi tạo;
- nếu `stage_2.ckpt` là thư mục Orbx, adapter sẽ restore run JAX trước đó.

Nếu bật `training.log_rae_latent_stats=true`, train loop JAX sẽ log `train_rae_latent_rms` và `train_rae_latent_var`. Nếu bật `training.log_activation_stats=true`, backend sẽ trả thêm intermediate feature của SiTDH để log RMS/variance cho output và từng block encoder/decoder, ví dụ `train_sitdh_output_rms`, `train_sitdh_act_enc_00_rms`, và `train_sitdh_act_dec_01_var`. Bù lại, các metric activation này có chi phí throughput và bộ nhớ rõ rệt hơn train loop mặc định. Nếu cần giảm độ trễ host-side, có thể override thêm `training.prefetch_factor` và `eval.prefetch_factor`; các key này được adapter forward thẳng sang DataLoader của đường JAX.

3.7. Wandb logging

Biến môi trường khuyến nghị:

```
export ENTITY=<wandb_entity>
export PROJECT=<wandb_project>
export WANDB_KEY=<wandb_api_key>
```

Đường JAX cũng chấp nhận các tên `WANDB_ENTITY` và `WANDB_API_KEY`. Adapter sẽ bridge các tên legacy ở trên nếu chúng đã được export sẵn. Với `src_jax/train.py`, mỗi workdir mới giờ sẽ ghi `wandb_run.json` để khóa đúng W&B run id của workdir đó. Run mới dùng `resume="never"`; các lần resume trực tiếp trên cùng workdir sau đó sẽ đọc lại file này, giữ đúng run id đã bind, rồi tự rewind W&B history về latest checkpoint_* step qua `resume_from`. Nhờ đó phần metric đã log sau checkpoint

cuối của session cũ sẽ không bị giữ lại sai lịch sử. Với train resume, runtime giờ sẽ xóa ngay thư mục `checkpoint_*` vừa restore khỏi ổ đĩa sau khi state đã vào bộ nhớ; trước mỗi lần save checkpoint mới nó cũng xóa các `checkpoint_*` cũ còn lại để workdir không cần chỗ cho hai Orbx checkpoint cùng lúc. Đường cleanup này giờ cũng chuẩn hóa cả Path lẫn workdir dạng chuỗi mà backend trainer truyền vào, nên resume DH/VAE không còn crash chỉ vì prune checkpoint với `str(workdir)`. Các notebook resume đóng gói sẵn trên branch learned source này không còn resume ngay trong workdir cũ nữa: chúng copy `checkpoint_*` mới nhất sang một workdir mới có timestamp, seed một `wandb_run.json` mới với run id mới, rồi giữ nguyên `exp_name` gốc để lineage dễ đọc trong khi history W&B của lần resume vẫn sạch. Adapter giờ cũng coi `train_step` là trục W&B chuẩn cho train, eval, FID, và media log, rồi mirror chính giá trị này sang internal Step của W&B để line plot không còn bị lệch khi đổi X axis. Cầu nối online FID cũng tắt các lệnh ghi W&B nội bộ từ backend và log lại cả EMA lẫn model FID trên cùng trục này, tránh cảnh báo step đi lùi kiểu `9999 < 10000` trên các run Kaggle dài.

3.8. FID trong train loop

Đường XLA hỗ trợ online FID trong `src/train.py`. Ví dụ:

```
eval:
  fid_ref: /path/to/reference_stats.npz
  fid_every: 25000
  fid_num_samples: 4096
  fid_per_proc_batch_size: 4
  fid_batch_size: 128
  fid_device: cpu
  fid_num_threads: 96
```

Đường JAX giờ cũng dùng block `eval` để chạy validation loss trên held-out ImageFolder, log `eval/ema_loss` mặc định và thêm `eval/model_loss` nếu bật `eval_model`. Với online FID, nên xây `fid_ref` bằng detector backend-native qua:

```
python3 src_jax/build_fid_stats.py \
  --input /path/to/reference_images \
  --output /path/to/reference_stats.pkl \
  --batch-size 64 \
  --num-workers 8
```

Trên Kaggle TPU, nên giữ `-num-workers` ở mức vừa phải (0-8 thường là đủ). Đường JAX này dùng worker kiểu spawn khi `num_workers > 0`, nên sạch hơn so với default fork.

Trên đường JAX, metric FID-4K (`cfg=...`) mặc định vẫn chằm EMA. Nếu bật `fid_eval_model`, adapter sẽ log thêm metric riêng FID-4K/model (`cfg=...`) cho online model để bạn đối chiếu trực tiếp với panel `network_samples`.

Riêng local trainer `src/train_stage1_rae.py` cũng có reconstruction FID cho validation reconstructions khi bật `eval.fid_ref`. Đường này chạy trên rank 0, log `val/fid`, `val/fid_num_samples`, và `val/fid_duration_sec`. `fid_every` ở đây được hiểu theo epoch, không phải optimizer step.

3.9. Sampling Stage 2

3.9.1. JAX một lệnh

```
python3 src_jax/sample.py \
  --config configs/stage2/sampling/ImageNet256/SiTDHXL-DINOv2-B_AG.yaml \
  --class-labels 207,360 \
  --output sample_jax.png
```

3.9.2. JAX distributed

```
python3 src_jax/sample_ddp.py \
  --config configs/stage2/sampling/ImageNet256/SiTDHXL-DINOv2-B.yaml \
  --sample-dir samples_jax \
  --num-samples 50000 \
  --label-sampling equal \
  --fid-ref /path/to/reference_stats.npz
```

Các config sampling JAX được commit dưới dạng template. Trước khi chạy sampling, cần tự điền `stage_2.ckpt` và, nếu dùng autoguidance, cả `guidance.guidance_model.ckpt` bằng checkpoint SiTDH tương thích.

3.10. Đẩy artifact lên Hugging Face

3.10.1. Lệnh độc lập

```
python3 src_jax/push_hf.py \
  --path results_jax/<run_name> \
  --repo-id <hf_user_or_org>/<repo_name>
```

3.10.2. Tích hợp ngay trong train

```
python3 src_jax/train.py \
  --config <config> \
  --data-path <train_root> \
  --hf-repo-id <hf_user_or_org>/<repo_name>
```

3.11. Notebook Kaggle TPU cho CelebA

Repo hiện ship notebook TPU `raes-jax-celeba-kaggle-tpuv5e8-sitdh-b-moe1.ipynb` để bám đúng flow Kaggle của branch `learned-source`:

- clone repo và checkout branch `jax-sit-dh-moe1`;
- đặt `UV_PROJECT_ENVIRONMENT=/tmp/.venv` và `UV_CACHE_DIR=/tmp/uv-cache`;
- chạy `uv sync -q` từ root của repo;
- chạy các bước phụ thuộc package qua `uv run`;
- chuẩn hoá dataset Kaggle CelebA thành `ImageFolder 256x256`;
- tạo bootstrap identity stat, build latent stat cho Stage 1;
- export reconstruction validation set, build `fid_ref`;
- build artifact diagonal GMM bằng `src_jax/build_source_gmm.py`;
- ghi sẵn config CelebA với block `source.enabled=true` cho `src_jax/train.py`.

Notebook này giữ dataset nguồn mặc định là bộ Kaggle CelebA chuẩn, center-crop và resize về 256x256, rồi materialize dữ liệu thành /kaggle/working/celeba256_imgfolder. Các notebook TPU dùng root ImageFolder và vẫn giữ eval.data_path trên split val. Các notebook TPU moe1 bật training.log_rae_latent_stats, training.log_activation_stats=true, và export RAE_JAX_REBUILD_BACKEND=1 để backend overlay luôn recopy interface sit_gmm_moe1 trong session Kaggle mới. Script src_jax/build_source_gmm.py trên branch này vẫn hỗ trợ full feature_extractor=flatten; với latent RAE 16 x 16 x 768, nó vẫn có thể giữ vector đầy đủ 196608 chiều khi fit GMM, đồng thời đi qua các safeguard mới của -storage-dtype auto và -compute-dtype auto để tránh quay lại đường float32-only và tự promote phần EM/KMeans offline về float32 nếu ai đó cố ép -compute-dtype float16. Tuy nhiên, bộ notebook Kaggle ship sẵn trên branch này giờ gọi builder một cách tường minh với -feature-extractor pyramid_16k và ghi artifact dạng *_source_gmm_pyr16k.npz, nên workflow mặc định không còn phụ thuộc vào full flatten nữa. spatial_mean vẫn là fallback tiết kiệm bộ nhớ khi ngay cả pyramid_16k cũng quá nặng với host.

Nếu chạy trên Kaggle TPU v5e-8, dùng notebook raes-jax-celeba-kaggle-tpuv5e8-sitdh-b-moe1.ipynb. Bản này cố định biến thể CelebA Stage 2 là SiTDH-B + moe1, sync dependency của repo vào /tmp/.venv, tự xóa cờ executable-stack mà Kaggle có thể chặn trên jaxlib, chạy bước xác nhận TPU trong một tiến trình Python mới, giữ Stage 1 và Stage 2 ở TPU, đồng thời build fid_ref bằng đúng detector backend dùng cho online FID, build thêm celeba256_source_gmm_pyr16k.npz qua -feature-extractor pyramid_16k, và giữ cadence checkpoint mặc định ở mốc 170000 step. Block source sinh ra dùng condition_dim=16, hidden_channels=256, router_temperature=2.0, balance_loss_weight=0.1, entropy_loss_weight=1.0e-2, và var_kl_loss_weight=1.0; các cell train/resume Stage 2 trên toàn bộ bộ notebook moe1 của branch này cũng đã thống nhất về training.num_workers=16, training.prefetch_factor, eval.prefetch_factor=4, training.log_rae_latent_stats=true, và training.log_activation_stats=true. Riêng các cell helper phía host đã hạ về src_jax/build_stage1_stats.py -batch-size 32 -num-workers 8, src_jax/build_fid_stats.py -num-workers 16, và src_jax/build_source_gmm.py -num-workers 16. Nếu cần sinh hàng loạt notebook Kaggle thay vì sửa tay từng bản, chạy python3 scripts/generate_ablation_notebooks.py -spec configs/ablation/celeba_sitdh_moe1_pyr16k.yaml -overwrite. Generator này ghi notebook vào generated_notebooks/sitdh_moe1_pyr16k/, cố định pyramid_16k cho toàn bộ ablation, chuyển secret wandb của notebook generated sang WANDB_Tung, và bơm trực tiếp -wandb-group/-wandb-tags để tất cả run cùng nằm dưới group celeba-sitdh-moe1-pyr16k-ablation. Cell PYCFG mà generator sinh ra cũng materialize sẵn các biến path như celeba_val_path trước khi ghi YAML, để script Python nhúng trong notebook luôn hợp lệ thay vì phụ thuộc vào biểu thức f-string có lồng dấu nháy. Spec commit sẵn hiện có các run 00 tới 07; run 07 mới dùng condition_dim=32 và hidden_channels=128.

Nếu đã có thư mục run Orbx của SiTDH-B + moe1 và cần train tiếp, dùng notebook raes-jax-celeba-kaggle-tpuv5e8-sitdh-b-moe1.ipynb. Notebook này đã được rút xuống thành bản resume-only cho trường hợp bắt đầu từ output archive của notebook cũ: cell đầu chạy lệnh unzip -o /kaggle/input/notebooks/kiuehongquan/rae-jax/_output_.zip -d /kaggle/working, sau đó mới kiểm tra repo đã có sẵn, chạy uv sync, fix jaxlib, lấy secret wandb, kiểm tra path dataset/FID/GMM, rồi tự tìm thư mục CelebA256_SiTDH-B_DINOv2-B_moe1_jax_tpuv5e8-* mới nhất bên dưới /kaggle/working/results_jax_tpu/, rồi mới tìm thư mục checkpoint_<step>/ mới nhất trong workdir đó, copy checkpoint này sang một workdir resume mới có timestamp, seed workdir mới bằng một wandb_run.json mới với run id mới, rồi mới truyền -workdir vào src_jax/train.py. Runtime tiếp tục restore latest_step() từ checkpoint mới nhất, xóa ngay checkpoint vừa restore trong workdir mới sau khi state đã vào RAM, và notebook vẫn không còn hardcoded cả thư mục run có timestamp lẫn checkpoint_100000. Artifact GMM mà notebook resume kiểm tra giờ là celeba256_source_gmm_pyr16k.npz.

Branch này cũng giữ song song bộ notebook TPU CelebA-HQ cho learned-source: raes-jax-celebahq-kaggle-tpuv5e8-sitdh-b-moe1.ipynb.

và `raes-jax-celebahq-kaggle-tpuv5e8-sitdh-b-moe1-resume.ipynb`. Chúng dùng chung `learned-source stack moe1` và `backend overlay`, nhưng tách riêng `flow chuẩn hoá dữ liệu`, `tên artifact`, và `mặc định run name` cho pipeline `CelebA-HQ`.

4. Cấu hình và ánh xạ sang JAX

4.1. Các block config chính

Tất cả entrypoint quan trọng đều dựa trên YAML của repo. Các block top-level:

- `stage_1`,
- `stage_2`,
- `transport`,
- `sampler`,
- `guidance`,
- `misc`,
- `training`,
- `eval`.

Local trainer `src/train_stage1_rae.py` còn đọc thêm hai block top-level: `gan` và `data`.

4.2. Ma trận script và block config

Block	<code>src_jax/train.py</code>	<code>src_jax/sample.py</code>	<code>src_jax/sample_eval.py</code>	<code>src/train_stage1_sample.py</code>
<code>stage_1</code>	yes	yes	yes	yes
<code>stage_2</code>	yes	yes	yes	no
<code>transport</code>	yes	yes	yes	no
<code>sampler</code>	yes	yes	yes	no
<code>guidance</code>	yes	yes	yes	no
<code>misc</code>	yes	yes	yes	no
<code>training</code>	yes	no	no	no
<code>eval</code>	yes	no	no	no

Local trainer Stage 1 mới cũng đọc YAML của repo:

Block	<code>src/train_stage1_rae.py</code>
<code>stage_1</code>	yes
<code>training</code>	yes
<code>gan</code>	yes
<code>data</code>	yes
<code>eval</code>	yes

4.3. Block stage_1

Adapter JAX không tạo schema mới cho Stage 1. Nó đọc trực tiếp:

- `encoder_config_path` hoặc `encoder_params.dinov2_path`,
- `pretrained_decoder_path`,
- `normalization_stat_path`,
- `encoder_input_size`.

Các giá trị này được chuyển sang encoder RAE của backend NNX để phục vụ reconstruction và decode latent khi sampling Stage 2.

Với `src/train_stage1_rae.py`, block `stage_1` còn quyết định:

- `stage_1.ckpt` có phải là checkpoint Stage 1 đầy đủ để finetune hay không;
- `pretrained_decoder_path` có được dùng làm warm start cho decoder hay không;
- DINOv2 có tiếp tục bị freeze hay được mở train thông qua `training.train_encoder`.

Nếu cần tính stat riêng cho dataset mới, có thể dùng:

```
python3 src_jax/build_stage1_stats.py \
  --config <stage1_config> \
  --input <train_imagefolder> \
  --output <stage1_stat.pt> \
  --set stage_1.params.normalization_stat_path=<bootstrap_identity_stat.pt>
```

Trong đó file bootstrap identity stat chỉ cần chứa `mean = 0` và `var = 1`. Mục tiêu là để pass tính stat đầu tiên không bị chuẩn hóa theo dataset khác, nhưng vẫn thỏa contract load stat của backend NNX.

4.4. Block stage_2

Một số key quan trọng:

- `target`: chỉ dùng để suy ra backbone NNX tương ứng;
- `ckpt`: có thể là file PyTorch `.pt` hoặc thư mục Orbx, nhưng trên branch này nó phải tương thích với shape của SiTDH;
- `input_size`, `patch_size`, `in_channels`;
- `hidden_size`, `depth`, `num_heads`: với SiTDH đây là các cặp encoder/decoder của backbone DH;
- `class_dropout_prob`: tỉ lệ label dropout cho CFG; với notebook CelebA-HQ một lớp trên đường JAX thì giá trị này được đặt về 0.0;
- các toggle như `use_rope`, `use_rmsnorm`, `use_swiglu`, `wo_shift`, `use_pos_embed`.

Với config mặc định của branch này, `target` sẽ là SiTDH và adapter JAX sẽ ánh xạ nó sang backend `lightning_ddt` trong khi vẫn giữ `interface_class=sit`. Các target DDT legacy cũng đi cùng đường `lightning_ddt`.

4.5. Block guidance

Hai mode chính:

- `cfg`: adapter dùng chính model làm conditional và unconditional pass, với nhãn null mặc định bằng `num_classes`;
- `autoguidance`: adapter load thêm `guidance.guidance_model` làm guide network.

4.6. Block training

Một số key được ánh xạ trực tiếp:

```
training:
  global_seed: 0
  epochs: 1400
  global_batch_size: 1024
  ema_decay: 0.9995
  num_workers: 4
  prefetch_factor: 2
  random_flip: true
  log_every: 100
  ckpt_every: 5000
  sample_every: 10000
  base_lr: 0.0002
  final_lr: 0.00002
  beta: [0.9, 0.95]
  wd: 0.0
  schedule_type: linear
  log_rae_latent_stats: false
  log_activation_stats: false
```

Lưu ý:

- nếu config chỉ cho `epochs`, adapter sẽ suy ra `total_steps` từ `num_train_samples`;
- optimizer mặc định vẫn là AdamW;
- scheduler được chuẩn hóa về các mode đơn giản như `constant`, `linear`, `cosine`;
- nếu `num_workers > 0`, có thể tăng `prefetch_factor` để queue trước nhiều batch hơn ở host-side PyTorch loader của đường JAX mà không đụng tới backend checkout;
- `random_flip` điều khiển việc chèn `RandomHorizontalFlip()` trước Stage 1 encode trên raw-image path; các config CelebA-HQ của branch này mặc định bật nó cho train nếu không override lại;
- nếu bật `log_rae_latent_stats=true`, đường JAX sẽ log `train_rae_latent_rms` và `train_rae_latent_var` cho latent RAE thực sự đi vào Stage 2;
- nếu bật `log_activation_stats=true`, đường JAX sẽ yêu cầu backend trả intermediate feature rồi log RMS/variance cho output của SiTDH và từng block encoder/decoder, ví dụ `train_sitdh_output_rms`, `train_sitdh_act_enc_00_rms`, và `train_sitdh_act_dec_01_var`.

Riêng `src/train_stage1_rae.py`, block training còn dùng thêm `batch_size`, `grad_accum_steps`, `image_size`, `precision`, `image_log_every`, `eval_every`, `save_every`, `recon_weight`, `train_encoder`,

`encoder_lr`, và `num_visuals`. Với DDP, effective batch size của local trainer là `batch_size * world_size * grad_accum_steps`.

4.7. Block gan

Đây là block dành riêng cho local trainer `src/train_stage1_rae.py`. Nó gồm:

- `disc.arch.dino_ckpt_path`: checkpoint backbone DINO cho discriminator;
- `disc.optimizer` và `disc.scheduler`: optimizer / LR schedule của discriminator;
- `disc.augment`: cấu hình DiffAug;
- `loss.perceptual_weight`: trọng số LPIPS;
- `loss.disc_start` và `loss.disc_upd_start`: epoch gate cho adversarial updates;
- `loss.max_d_weight`: ngưỡng clamp cho adaptive discriminator weight phía generator.

4.8. Block data

Local trainer Stage 1 dùng `data.train_path` làm root train `ImageFolder`; validation đi qua `eval.data_path`. Nếu thư mục không có class subdirectory, trainer vẫn có thể wrap chúng thành dataset một lớp.

4.9. Block source

Trên branch `jax-sit-dh-moe1`, block source bật đường learned-source GMM + CNN-MoE. Khi `source.enabled=true`, adapter sẽ đổi `interface_class` từ `sit` sang `sit_gmm_moe1`.

```
source:
  enabled: true
  kind: gmm_moe1
  gmm_stats_path: artifacts/celeba256_source_gmm_pyr16k.npz
  num_modes: 4
  condition_dim: 16
  hidden_channels: 256
  router_temperature: 2.0
  soft_moe: true
  balance_loss_weight: 0.1
  entropy_loss_weight: 1.0e-2
  var_kl_loss_weight: 1.0
  target_variance: 1.0
  logvar_min: -8.0
  logvar_max: 4.0
  var_floor: 1.0e-5
  posterior_eps: 1.0e-6
  weight_prior: 1.0e-2
  em_iters: 100
  em_tol: 1.0e-4
  em_restarts: 3
  dead_count_threshold: 1.0
  active_mode_fraction_threshold: 0.01
```

- `gmm_stats_path` trỏ tới artifact `.npz` được build bởi `src_jax/build_source_gmm.py`; script này cũng lưu luôn cách trích feature cho GMM. Trên branch DH, mode mặc định giờ là `flatten`: latent

RAE $16 \times 16 \times 768$ được giữ nguyên thành vector 196608 chiều khi fit GMM. Builder vẫn preallocate đúng một ma trận $N \times D$, nhưng `-storage-dtype auto` sẽ tự chọn float16 cho full-latent DH để giảm gần một nửa RAM host, còn `-compute-dtype auto` sẽ upcast từng chunk EM/KMeans về float32. Khi cần feature nén có cấu trúc hơn thì vẫn có thể ép pyramid_16k; chỉ dùng spatial_mean khi cần fallback tiết kiệm RAM;

- `num_modes` là số component GMM đồng thời là số expert của MoE;
- `condition_dim` là chiều embedding mode trước khi broadcast bias vào trunk CNN;
- `hidden_channels` là độ rộng trunk/source experts; với latent RAE CelebA-HQ có shape $B \times 16 \times 16 \times 768$, notebook mặc định tăng lên 256 thay vì giữ mức hẹp của flow VAE;
- `router_temperature` và `soft_moe` điều khiển độ sắc của router và việc dùng gating mềm hay straight-through;
- `balance_loss_weight`, `entropy_loss_weight`, và `var_kl_loss_weight` là các regularizer phụ cộng vào objective flow-matching chính;
- `weight_prior`, `em_iters`, `em_tol`, `em_restarts`, `dead_count_threshold`, và `active_mode_fraction_thresh` phải khớp recipe đã dùng để build artifact GMM.

4.10. Block eval

Đường JAX giờ dùng block eval cho cả validation loss lẫn online FID. Các key `data_path`, `eval_every`, `batch_size`, `num_workers`, `prefetch_factor`, `random_flip`, `max_batches`, và `eval_model` sẽ điều khiển held-out validation loop trong `src_jax/train.py`. Với `fid_ref`, nên ưu tiên file .pkl/.pickle được build bởi `src_jax/build_fid_stats.py`; nếu đưa file .npz cũ vào, adapter vẫn tự chuyển nó sang định dạng pickle mà backend NNX hiểu được. Trên đường JAX, metric FID-4K (`cfg=...`) mặc định chấm EMA; nếu bật `fid_eval_model`, adapter sẽ log thêm FID-4K/model (`cfg=...`) cho online model mà không thay metric EMA mặc định.

Trên local trainer `src/train_stage1_rae.py`, block eval dùng `data_path`, `batch_size`, `num_workers`, và `max_batches` cho validation reconstructions. Nếu cung cấp thêm `fid_ref`, trainer còn chạy reconstruction FID với các key dùng chung `fid_every`, `fid_batch_size`, `fid_device`, và `fid_num_threads`; ở đường này `fid_every` được tính theo epoch. Các key tùy chọn như `max_batches` và `fid_num_threads` có thể để trống hoặc ghi null.

4.11. Override từ CLI

Các endpoint JAX cho phép override nhanh bằng:

```
python3 src_jax/train.py \
  --config <config> \
  --set training.global_batch_size=256 \
  --set training.prefetch_factor=8 \
  --set eval.prefetch_factor=4 \
  --set training.log_rae_latent_stats=true \
  --set training.log_activation_stats=true \
  --set guidance.scale=1.5
```

Mục tiêu là giữ đúng một schema YAML, nhưng vẫn cho phép chỉnh nhanh như CLI.

5. Vận hành, artifact và FID

5.1. Sampling Stage 2 trên đường JAX

5.1.1. Sampling nhanh

```
python3 src_jax/sample.py \
  --config <sample_config> \
  --seed 42 \
  --class-labels 207,360 \
  --output sample_jax.png
```

5.1.2. Sampling phân tán

```
python3 src_jax/sample_ddp.py \
  --config <sample_config> \
  --sample-dir samples_jax \
  --num-samples 50000 \
  --per-proc-batch-size 4 \
  --label-sampling equal \
  --fid-ref /path/to/reference_stats.npz
```

Artifact đầu ra:

- các file PNG theo index;
- các shard .npz theo rank nếu bật `-save-npz`;
- giá trị FID in ra terminal nếu cung cấp `-fid-ref`.

5.2. Build reference statistics cho FID

```
python src/build_fid_stats.py \
  --input /path/to/reference_images \
  --output /path/to/reference_stats.npz \
  --device cpu \
  --batch-size 64 \
  --num-workers 32 \
  --num-threads 96
```

Đây là đường `torch-fidelity` chung cho nhánh XLA/PyTorch. Nếu cần `fid_ref` cho online FID ở JAX, nên build bằng detector backend-native:

```
python3 src_jax/build_fid_stats.py \
  --input /path/to/reference_images \
  --output /path/to/reference_stats.pkl \
  --batch-size 64 \
  --num-workers 8
```

Trên host JAX như Kaggle TPU, nên giữ `-num-workers` ở mức vừa phải (0-8 thường là đủ). Script này dùng worker kiểu `spawn` khi `num_workers > 0`, nên tránh được cảnh báo `os.fork()` thường gặp với JAX đa luồng.

5.3. Artifact của Stage 2 training

5.3.1. Đường XLA

Một experiment trong results/ thường có:

- log.txt,
- checkpoints/<step>.pt,
- fid_eval/step_<step>/... nếu bật online FID.

5.3.2. Đường JAX

Một run trong results_jax/ thường có:

- jax_adapter_config.json,
- thư mục checkpoint Orbx do backend quản lý,
- media và scalar trên wandb nếu bật -wandb,
- toàn bộ workdir có thể được đẩy lên Hugging Face nếu truyền -hf-repo-id.

5.4. Đẩy model lên Hugging Face

```
python3 src_jax/push_hf.py \
  --path results_jax/<run_name> \
  --repo-id <hf_user_or_org>/<repo_name>
```

Hoặc gán thẳng vào lệnh train:

```
python3 src_jax/train.py \
  --config <config> \
  --data-path <train_root> \
  --hf-repo-id <hf_user_or_org>/<repo_name>
```

5.5. Các lưu ý vận hành quan trọng

- Đường JAX cố ý giữ nguyên schema YAML của repo để giảm chi phí bảo trì.
- stage_2.ckpt trên đường JAX có thể là PyTorch hoặc Orbx, nhưng cách load sẽ khác nhau.
- Đường JAX giờ cũng có validation loss định kỳ qua block eval, ngoài online FID.
- FID trên đường JAX nên dùng reference stats build bởi src_jax/build_fid_stats.py để cùng detector với backend; nếu cần, adapter vẫn chuyển .npz cũ sang pickle nội bộ.
- Online FID ở JAX mặc định chấm EMA. Nếu cần đối chiếu với model hiện thời, bật eval.fid_eval_model=true để log thêm metric FID-4K/model (cfg=...) bên cạnh series EMA mặc định.
- Nếu chỉ cần inference Stage 1 ở JAX, dùng src_jax/stage1_sample.py.
- Nếu cần stat Stage 1 hoặc reconstruction cả thư mục ở JAX, dùng src_jax/build_stage1_stats.py và src_jax/reconstruct_folder.py. Các lệnh này có thể dùng YAML chỉ có stage_1, không cần stage_2.target.

- Nếu cần huấn luyện Stage 1 kiểu GAN đầy đủ trên GPU, dùng `src/train_stage1_rae.py` hoặc hai notebook Kaggle GPU `tunedinov2-stage1-scratch-kaggle.ipynb` và `tunedinov2-stage1-finetune-dinov`. Đường này hiện là local PyTorch/CUDA, chưa có bản TPU/XLA riêng.
- Nếu chạy trên Kaggle với CelebA cho branch `learned-source` này, bắt đầu từ notebook TPU `raes-jax-celeba-kaggle` để giữ đúng flow clone repo, đặt `UV_PROJECT_ENVIRONMENT=/tmp/.venv` và `UV_CACHE_DIR=/tmp/uv-cache`, chạy `uv sync -q`, tải dataset Kaggle CelebA, center-crop và resize sang `ImageFolder`, rồi chạy các bước phụ thuộc package qua `uv run`. Flow notebook này giờ build thêm artifact `celeba256_source_gmm.npz` qua `src_jax/build_source_gmm.py`. Trên latent RAE DH lớn, script này giờ mặc định giữ `feature_extractor=f` tức fit GMM trực tiếp trên vector đầy đủ 196608 chiều. Builder vẫn `preallocate` đúng một ma trận $N \times D$, nhưng `-storage-dtype auto` sẽ chọn `float16` cho full-latent DH và `-compute-dtype auto` sẽ upcast từng chunk fit về `float32`; nếu có ai ép `-compute-dtype float16` cho full flatten quá rộng thì fitter cũng sẽ tự promote phần KMeans/EM offline về `float32`. Nếu muốn feature nén có cấu trúc hơn thì có thể ép sang `pyramid_16k`, còn `spatial_mean` chỉ nên dùng khi thật sự thiếu RAM, rồi mới ghi config Stage 2 với block `source.enabled=true`. Route TFDS hay Hugging Face cho CelebA-HQ vẫn còn trong repo, nhưng không còn là flow mặc định của branch này.
- Nếu phần cứng là Kaggle TPU `v5e-8`, dùng notebook `raes-jax-celeba-kaggle-tpuv5e8-sitdh-b-moe1.ipynb` bản này cố định biến thể Stage 2 là SiTDH-B + `moe1`, chuyển FID và build reference stats sang detector backend-native của JAX, đồng thời cũng sync dependency của repo vào `/tmp/.venv` trước các bước `uv run`, tự xóa cờ `executable-stack` mà Kaggle có thể chặn trên `jaxlib`, rồi kiểm tra `jax/TPU` bằng tiến trình Python mới. Notebook này cũng chuẩn hoá CelebA sang `ImageFolder` trước khi build Stage 1 stat, build `fid_ref`, build `celeba256_source_gmm.npz`, export `RAE_JAX_REBUILD_BACKEND=1`, và giữ mốc checkpoint mặc định ở 210000 step.
- Nếu cần cùng flow Kaggle TPU đó nhưng muốn train biến thể SiTDH-B + `moe1`, dùng notebook `raes-jax-celeba-kaggle-tpuv5e8-sitdh-b-moe1.ipynb`. Bản này giữ toàn bộ fix Kaggle/JAX hiện có, đổi Stage 2 sang recipe DH/two-tower SiTDH-B với `hidden_size=[768, 2048]`, `depth=[12, 2]`, `num_heads=[12, 16]`, `use_pos_embed=true`, `class_dropout_prob=0.0`, ghi config `CelebA256_SiTDH-B` build artifact GMM trước train, và dùng cùng cadence checkpoint 210000 step.
- Nếu đã có sẵn một run Orbax của SiTDH-B + `moe1` và cần train tiếp từ checkpoint hiện có, dùng notebook `raes-jax-celeba-kaggle-tpuv5e8-sitdh-b-moe1-resume.ipynb`. Notebook này là bản resume-only tối giản cho trường hợp bắt đầu từ output archive của notebook cũ: cell đầu chạy lệnh `unzip -o /kaggle/input/notebooks/kieuhongquan/rae-jax/_output_.zip -d /kaggle/working`, sau đó chỉ kiểm tra repo đã có sẵn, chạy `uv sync`, lấy secret wandb, kiểm tra path dataset/FID/GMM, export `RAE_JAX_REBUILD_BACKEND=1`, rồi tự tìm thư mục run CelebA256_SiTDH-B_DINOV2-B_moe1_jax_tpuv5e8 mới nhất bên dưới `/kaggle/working/results_jax_tpu/` và thư mục `checkpoint_<step>/` mới nhất trong workdir đó, copy checkpoint này sang một workdir resume mới có timestamp, seed workdir mới bằng một `wandb_run.json` mới với run id mới, rồi mới truyền `-workdir` vào `src_jax/train.py`. Runtime cũng tiếp tục restore bằng `latest_step()`, xóa ngay checkpoint vừa restore trong workdir mới sau khi state đã vào RAM, nên notebook này không còn hardcoded `checkpoint_100000`.
- Phần huấn luyện Stage 1 kiểu GAN vẫn chưa được JAX hóa trong branch này.

5.6. Checklist thực tế trước khi chạy dài

1. Xác nhận `stage_1` và `stage_2` khớp latent shape.
2. Xác nhận dữ liệu train và val đúng dạng `ImageFolder`.

3. Với dataset mới, build lại `stage1_stat.pt` thay vì tái dùng stat của ImageNet nếu muốn latent normalization khớp dữ liệu hơn.
4. Build `fid_ref` trước nếu muốn FID ổn định.
5. Export đủ biến wandb trước khi bật `-wandb`.
6. Nếu muốn upload lên Hugging Face, xác nhận token nằm trong biến môi trường được chỉ định bởi `-hf-token-env`.
7. Với JAX sampling số lượng lớn, chọn `-per-proc-batch-size` đủ an toàn theo bộ nhớ thiết bị.